IntentChecker: Validating Developer Intentions in Solidity via Intent Annotations to Mitigate Numeric Logic error

Inseong Jeon[1], Sundeuk Kim[1], Hyunwoo Kim[1], Hoh Peter In[1*]

[1*]Department of Computer Science, Korea University, 145, Anam-ro, Seonbuk-gu, 02841, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): hoh_in@korea.ac.kr;
Contributing authors: iwwyou@korea.ac.kr; sd_kim@korea.ac.kr;
khw0809@korea.ac.kr;

Abstract

Logic errors in smart contracts—where program behavior diverges from developer intent—are among the most challenging vulnerabilities to detect. Unlike well-defined patterns such as reentrancy or integer overflow, logic errors require knowledge of developer intent, which is absent from the code itself. Existing approaches, including static analyzers, dynamic testing, and LLM-based methods, cannot reliably identify such errors without explicit intent representation. This paper presents IntentChecker, a tool that enables developers to specify and validate their intentions for numeric operations during smart contract development. IntentChecker introduces an intent annotation model with two annotation types: `@During` annotations specify intended relationships at specific program points within a function, while `@Post` annotations specify intended relationships after function execution. The underlying analysis uses interval-domain abstract interpretation to compute value ranges at each program point and check whether they satisfy the specified intentions. We evaluated IntentChecker on real-world smart contracts, demonstrating its effectiveness in validating developer intentions and providing immediate feedback during development.

Keywords: Smart Contract Development, Solidity, Numeric Logic Error, Intent Annotations, Abstract Interpretation, Developer Tools

1 Introduction

Smart contracts are immutable once deployed, making security assurance during development critical. Among vulnerability types, logic errors present a distinct challenge: they arise when program behavior diverges from developer intent, a discrepancy that is inherently invisible to compilers and external reviewers alike (**?**).

A recent empirical study (**?**) confirms this challenge in practice. Practitioners identified logic errors as the most difficult vulnerability type to detect, reporting that existing security tools are least effective against them. This gap exists because current approaches—static analyzers, dynamic testing, and even recent LLM-based methods—predominantly target well-defined vulnerability patterns. Logic errors, however, require knowledge of developer intent, which is absent from the code itself. Various research efforts have attempted to detect specific logic errors, but without explicit intent representation, they must rely on predefined detection patterns. Such patterns can identify violations of specific rules, but not actual deviations from intended behavior. Consequently, even when these tools report no issues, developers cannot be certain that their code behaves as intended.

(**?**) also provides insight into developer needs. Developers expressed a preference for development-integrated tools such as IDE extensions and linters over standalone analyzers. Moreover, when asked about the most valued properties of security tools, developers prioritized low false negatives (92.3%) over low false positives (50%), indicating a strong preference for tools that do not miss real issues, even at the cost of occasional false alarms. This suggests a need for development-phase tools that can provide sound guarantees about code behavior. To determine where such intent specification is most needed, we analyzed 1,023 non-trivial functions from DAppSCAN-source (**?**) (see Table **??**). Numeric variable types appear in 82% of these functions, with `uint` alone used in 78%. Since numeric computations form the core of smart contract business logic —handling token transfers, fee calculations, and balance updates—explicitly specifying developer intent for these operations becomes essential.

Based on these observations, this paper presents IntentChecker, a tool that enables developers to specify and validate their intentions for numeric operations during the development phase. IntentChecker introduces an intent annotation model consisting of two annotation types: `@During` annotations specify intended relationships that should hold at specific program points within a function, while `@Post` annotations specify intended relationships that should hold after the function completes execution. Developers write these annotations alongside their code, and IntentChecker validates whether the program's abstract execution satisfies these specified intentions.

The underlying value analysis is performed using interval-domain abstract interpretation, building upon our previous work on SolQDebug (**?**), an interactive debugger for Solidity. Through `@Debugging` annotations, developers specify value ranges for variables, and the analyzer computes abstract values at each program point. IntentChecker extends this capability by checking whether these computed values satisfy the developer-specified intent annotations, providing immediate feedback during development rather than requiring completed code.

This paper makes the following contributions:

- We present an intent annotation model that enables developers to express intended numeric relationships at both statement-level (`@During`) and function-level (`@Post`) granularity.
- We develop validation algorithms for each annotation rule that leverage interval-domain abstract interpretation to check whether program behavior satisfies specified intentions.
- We evaluate IntentChecker on real-world smart contracts, demonstrating its effectiveness in validating developer intentions during the development phase.

The remainder of this paper is organized as follows. Section **??** provides background on numeric logic errors and their classification by execution semantics. Section **??** describes the design of IntentChecker, including its intent model and validation engine. Section **??** presents our evaluation results. Section **??** discusses limitations and future work. Section **??** reviews related work, and Section **??** concludes.

2 Background

2.1 Numeric Logic Errors in Solidity

Recent research has identified various numeric-related weakness in smart contracts (**?????**). Building on these findings, numeric logic errors can be defined as follows:

Definition 1 (Numeric Logic Error) Bugs that allow transactions to complete successfully without compile-time or runtime exceptions, but violate the developer's intended numerical or state relationships during function execution.

These errors can manifest in different execution contexts. Single-transaction errors occur within the execution of a single function call—where computation, function calls, and output generation happen in one atomic sequence. In contrast, multiple-transaction errors only emerge through sequences of function calls, requiring analysis of inter-function dependencies and contract-level invariants. Examples of multiple-transaction errors include Risky First Deposit and Price Oracle Manipulation (**??**), where attackers exploit state changes across multiple function invocations.

This paper focuses on single-transaction function execution, as this aligns with the development-phase validation model where developers can immediately verify individual function behaviors against their intentions. The representative single-transaction numeric logic error patterns identified in prior work are summarized below:

- **Div In Path** (**?**): Division results used in conditions can lead to unexpected control flow due to integer truncation.
- **Operator Order Issue** (**?**): Arithmetic order matters: `a/b*c` $\neq$ `a*c/b` due to intermediate truncation.
- **Exchange Problem** (**?**): Rounding errors in exchange calculations may result in zero output despite valid input, or receiving tokens without payment.

- **Precision Loss Trend** (**?**): Integer division's inherent truncation tends to favor one side, potentially misaligning with protocol intent.
- **Minor Amount Retention** (**?**): Integer division in distributions leaves remainders unallocated to any recipient, causing permanent lock.
- **Wrong Interest Rate Order** (**?**): Balance calculations using outdated or prematurely updated interest rates yield incorrect results.
- **Wrong Checkpoint Order** (**?**): Incorrect ordering between state updates and checkpoints leads to reward miscalculation.
- **Erroneous Accounting** (**?**): Incorrect implementation of business model formulas introduces small errors that accumulate into substantial losses.
- **Greedy Contract** (**?**): Contract can receive funds but lacks proper logic to withdraw them, causing permanent lock.

Despite their diversity, these error patterns share a common root: the actual behavior diverges from what the developer intended.

2.2 Motivation for Intent-Based Validation

Existing approaches to detecting numeric logic errors—whether pattern-based static analyzers, formal verification tools, or LLM-based methods—fundamentally analyze code without access to the developer's actual intentions. Since logic errors arise when program behavior diverges from developer intent, and this intent exists only in the developer's mind, these approaches cannot reliably distinguish between intended behavior and unintended bugs. To bridge this gap, developers must be able to *explicitly specify* their intended numeric behaviors. The key question then becomes: how can we systematically represent numeric intentions within a single function execution?

Numeric variables flow through three distinct stages during function execution, and developers may have specific intentions at each stage:

Fig. 1: Variable Flow Stages in Function Execution

- **COMPUTE**: Developers intend that arithmetic operations produce results within expected ranges, and that variable values change (or remain unchanged) as expected after specific statements.
- **TRANSFER**: Developers intend that function calls receive arguments satisfying certain constraints, ensuring that called functions operate on valid inputs.
- **OUTPUT**: Developers intend that functions return expected values and update state variables correctly, reflecting the intended effects of the computation.

To validate that this classification comprehensively covers numeric logic errors, the previously identified error patterns are mapped to these variable flow stages:

Table 2: Mapping of Error Patterns to Variable Flow Stages

Variable Flow	Error Pattern
COMPUTE	Operator Order Issue Precision Loss Trend Div In Path Exchange Problem Wrong Interest Rate Order Erroneous Accounting
TRANSFER	Minor Amount Retention
OUTPUT	Greedy Contract Wrong Checkpoint Order

This classification provides a foundation for designing an intent specification mechanism that can address errors at each stage of variable flow.

2.3 IntentChecker Overview

Based on the variable flow classification, we propose IntentChecker, a framework that enables developers to declare their intended numeric behaviors and validates them through static analysis. IntentChecker consists of two core components: an *Intent Model* for specifying developer intentions, and a *Validation Engine* for checking whether code behavior conforms to those intentions. The intent model provides two annotation types aligned with the variable flow stages: `@During` annotations specify intended relationships at specific program points within a function (covering COMPUTE and TRANSFER stages), while `@Post` annotations specify intended relationships after function execution (covering OUTPUT stage and function-level invariants). Developers express their intentions using a lightweight DSL embedded in comments, allowing intent specifications to coexist with existing code without modification.

The validation engine employs interval-based abstract interpretation to compute the actual value ranges of variables, then compares them against the declared intent. The validation result is deterministically categorized into one of three states:

$$\textbf{Satisfied} \iff I_{\text{actual}} \subseteq I_{\text{intent}}$$

$$\textbf{Violated} \iff I_{\text{actual}} \cap I_{\text{intent}} = \varnothing$$

$$\textbf{Uncertain} \iff I_{\text{actual}} \not\subseteq I_{\text{intent}} \ \wedge \ I_{\text{actual}} \cap I_{\text{intent}} \neq \varnothing$$

If neither fully satisfied nor clearly violated, the system reports `Uncertain`, along with a confidence score representing the probability of satisfaction and the specific sub-intervals where violation may occur.

To perform this interval-based validation, IntentChecker builds upon SolQDebug, a source-level Solidity debugger designed for interactive abstract interpretation during code authoring. SolQDebug constructs a dynamic control-flow graph from source

code and executes abstract interpretation over a fixed-point lattice of interval values. While SolQDebug was originally designed for single-transaction, single-contract analysis, IntentChecker extends this foundation to support multiple contracts defined within the same source file. Additionally, commonly used libraries are pre-analyzed and their semantics are imported during validation, enabling more precise analysis without requiring repeated computation.

The detailed design of the intent model and validation algorithms are presented in Section **??**.

3 The Design of IntentChecker

3.1 System Architecture

3.2 Intent Model

3.3 Validation Engine

4 Evaluation

5 Discussion

6 Related Works

6.1 Numeric Logic Error Detection in Solidity

Logic error detection focuses on identifying discrepancies between developer intentions and actual program behavior in smart contracts.

(**?**) addresses business logic violations in smart contracts by using Dynamic Condition Response (DCR) graphs to model expected behaviors including temporal constraints, role-based access control, and inter-function dependencies. The system monitors runtime transactions against user-provided DCR models and their function mappings, generating violation reports when execution activities deviate from the specified business logic patterns. (**?**) verifies temporal properties of smart contracts by accepting user-specified Linear Temporal Logic (LTL) formulas and attack models to check both safety and liveness properties. The system not only detects violations but also generates concrete attack scenarios that demonstrate how the temporal properties can be violated, providing developers with actionable insights into potential logic errors. (**?**) enables developers to specify function and address behaviors using the ConSol syntax alongside their Solidity code, then verifies whether the implementation adheres to these behavioral specifications and generates behavior-compliant Solidity programs without annotations. (**?**) leverages multimodal learning to automatically infer invariants and detect machine unauditable bugs (MUBs) (**?**) by analyzing both source code and natural language artifacts including comments, function signatures, and documented transaction scenarios. The system employs a fine-tuned model with Tier of Thought (ToT) prompting that progressively predicts transaction contexts, generates and ranks invariants, and ultimately identifies vulnerabilities through three-tier reasoning. (**?**) proposes a novel type system for Solidity variables defined as tuples

of ⟨Financial Type, Scaling Factor, Token Unit, Address⟩ to detect accounting errors through type checking. The system reinterprets Solidity code based on detection patterns such as incorrectly adding balances with fees, enabling systematic identification of numeric logic errors that arise from mismatched financial operations.

Furthermore, there are also approaches that address multiple transaction logic errors such as price manipulation (**?????**) and flash loan attacks (**??**).

In contrast to existing approaches, IntentChecker operates within the debugging context to provide immediate feedback on code under development, fundamentally differing from tools that only verify completed code. Through `@During` annotations, it uniquely enables validation of intermediate execution states within functions, allowing developers to verify their step-by-step intentions. Moreover, rather than being limited to specific patterns, IntentChecker comprehensively validates all numeric operations using interval-domain abstract interpretation for sound verification, achieving both generality and reliability simultaneously.

6.2 Specification-based Smart Contract Verification

Specification-based verification approaches focus on verifying general properties and correctness requirements of smart contracts rather than specific developer intentions.

(**?**) provides a framework that transforms developer annotations into Solidity assertion code, enabling testing with existing tools like MythX (**?**) and Mythril (**?**). The framework supports multiple annotation types including if_succeed (conditions for successful function returns), invariant (contract-level properties that must always hold), if_updated (conditions checked when state variables change), and assert (validations at specific execution points), making formal specifications accessible through familiar testing pipelines. (**?**) builds upon Scribble (**?**) to verify whether Solidity code satisfies specifications written in the Scribble language, leveraging C-based verification tools like SeaHorn (**?**), libFuzzer (**?**), and KLEE (**?**) by transforming smart contracts into LLVM-IR based harnesses. The framework abstracts the 2^{160} possible user addresses into a small set of representative users (implicit, transient, and explicit), enabling verification of properties that must hold for arbitrary numbers of users, such as accounting properties like "the contract balance equals the sum of all user deposits." (**?**) verifies Solidity code using two types of annotations: specification annotations (loop invariants, call invariants, and call modifies) that developers declare as assumptions, and verification annotations (ensures, reverts_if, and contract invariants) that the tool checks against the implementation. The system assumes the truth of specification annotations and uses them to verify whether the code satisfies function post-conditions, correctly handles revert conditions, and maintains contract invariants throughout execution. (**?**) focuses on verifying liquidity properties of smart contracts through formal methods by converting both the contract code and user-defined liquidity properties into SMT constraints. The system then checks whether the contract satisfies these properties using SMT solvers, generating counterexamples when violations are detected to help developers understand potential liquidity issues.

While specification-based approaches focus on verifying high-level contract properties and invariants in completed code, IntentChecker addresses a fundamentally

different problem: validating specific developer intentions about numeric values during the debugging process. Unlike existing tools that treat library functions as black boxes, IntentChecker performs interval-domain analysis that tracks actual value ranges through library function calls (e.g., SafeMath operations), enabling developers to verify their step-by-step numeric intentions rather than abstract contract-level properties.

6.3 Tools for Strengthening Security During Development Phase

Development-phase security tools integrate vulnerability detection directly into the coding workflow, enabling developers to identify and address security issues as they write code rather than in post-development analysis. (**?**) presents an integrated development environment for smart contract development that combines natural language model-based code autocompletion with security validation. The platform integrates existing static analysis tools including Oyente (**?**), Mythril (**?**), and Securify (**?**) to provide comprehensive security reports during development workflow. (**?**) provides a Solidity linter that detects predefined vulnerability patterns including low-level calls, reentrancy, and deprecated function usage during development. The tool enforces security best practices by checking for patterns such as proper visibility declarations, send result validation, and secure compiler version usage.

While these development-phase tools rely on predefined detection patterns for known vulnerabilities, IntentChecker enables developers to specify their intended numeric behaviors through annotations and validates whether the actual program execution matches these specified intentions.

7 Conclusion

References

Arora, S., et al.: SecPLF: Secure protocols for loanable funds against oracle manipulation attacks. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIACCS), pp. 1394–1405 (2024). https://doi.org/10.1145/3634737.3637681

Bartoletti, M., et al.: Solvent: liquidity verification of smart contracts. In: International Conference on Integrated Formal Methods (iFM), pp. 256–266 (2024). https://doi.org/10.1007/978-3-031-76554-4$_1$4

Cadar, C., et al.: KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In: OSDI, pp. 209–224 (2008). https://doi.org/10.5555/1855741.1855756

Chaliasos, S., et al.: Smart contract and defi security tools: Do they meet the needs of practitioners?. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 1–13 (2024). https://doi.org/10.1145/3597503.3623302

Chen, J., et al.: NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. IEEE Transactions on Software Engineering (2025). https://doi.org/10.1109/TSE.2025.3555622

Consensys Diligence: Mythril. https://github.com/ConsenSysDiligence/mythril (2025). Accessed December 2025

Consensys Diligence: MythX. https://mythx.io/ (2025). Accessed December 2025

Consensys Diligence: Scribble. https://diligence.security/scribble/ (2025). Accessed January 2026

Eshghie, M., et al.: Highguard: Cross-chain business logic monitoring of smart contracts. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 2378–2381 (2024). https://doi.org/10.1145/3691620.3695356

Gurfinkel, A., et al.: The SeaHorn verification framework. In: International Conference on Computer Aided Verification, pp. 343–361 (2015). https://doi.org/10.1007/978-3-319-21690-4$_2$0

Jeon, I., et al.: SolQDebug: Debug Solidity Quickly for Interactive Immediacy in Smart Contract Development (2025). https://doi.org/10.21203/rs.3.rs-8077153/v1

Kong, Q., et al.: Defitainter: Detecting price manipulation vulnerabilities in defi protocols. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 1144–1156 (2023). https://doi.org/10.1145/3597926.3598124

Li, X., et al.: Penetrating the hostile: Detecting defi protocol exploits through cross-contract analysis. IEEE Transactions on Information Forensics and Security 20, 11759–11774 (2025). https://doi.org/10.1109/TIFS.2025.3626979

LLVM: libFuzzer. https://llvm.org/docs/LibFuzzer.html (2025). Accessed December 2025

Luu, L., et al.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 254–269 (2016). https://doi.org/10.1145/2976749.2978309

Nguyen, T.D., et al.: An idealist's approach for smart contract correctness. In: International Conference on Formal Engineering Methods, pp. 11–28 (2023). https://doi.org/10.1007/978-981-99-7584-6$_2$

Protofire: Solhint. https://github.com/protofire/solhint (2025). Accessed Janauary 2026

Ramezany, S., Setthawong, R., Setthawong, P.: Midnight: An Efficient Event-driven EVM Transaction Security Monitoring Approach For Flash Loan Detection. In: Proceedings of the 2023 20th International Joint Conference on Computer Science and Software Engineering (JCSSE), pp. 237–241 (2023). https://doi.org/10.1109/JCSSE58229.2023.10201970

Ren, M., et al.: Making smart contract development more secure and easier. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), pp. 1360–1370 (2021). https://doi.org/10.1145/3468264.3473929

Soud, M., Liebel, G., Hamdaqa, M.: A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses. Empirical Software Engineering 29(1), 13 (2024). https://doi.org/10.1007/s10664-023-10398-5

Stephens, J., et al.: SmartPulse: automated checking of temporal properties in smart contracts. In: Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP), pp. 555–571 (2021). https://doi.org/10.1109/SP40001.2021.00085

Sun, Y., et al.: Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). https://doi.org/10.1145/3597503.3639117

Tsankov, P., et al.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 67–82 (2018). https://doi.org/10.1145/3243734.3243780

Wang, D., et al.: Defiguard: A price manipulation detection service in defi using graph neural networks. IEEE Transactions on Services Computing (2024). https://doi.org/10.1109/TSC.2024.3489439

Wang, S.J., Pei, K., Yang, J.: Smartinv: Multimodal learning for smart contract invariant inference. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 2217–2235 (2024). https://doi.org/10.1109/SP54263.2024.00126

Wei, G., et al.: Consolidating Smart Contracts with Behavioral Contracts. Proceedings of the ACM on Programming Languages 8(PLDI), 965–989 (2024). https://doi.org/10.1145/3656416

Wesley, S., et al.: Verifying Solidity smart contracts via communication abstraction in SmartACE. In: International Conference on Verification, Model Checking, and Abstract Interpretation, pp. 425–449 (2022). https://doi.org/10.1007/978-3-030-94583-$1_2$1

Wu, S., et al.: Defiranger: Detecting defi price manipulation attacks. IEEE Transactions on Dependable and Secure Computing 21(4), 4147–4161 (2023). https://doi.org/10.1109/TDSC.2023.3346888

Xi, R., Wang, Z., Pattabiraman, K.: POMABuster: Detecting Price Oracle Manipulation Attacks in Decentralized Finance. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 3923–3942 (2024). https://doi.org/10.1109/SP54263.2024.00257

Zhang, B.: Towards finding accounting errors in smart contracts. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). https://doi.org/10.1145/3597503.3639128

Zhang, W., et al.: Nyx: Detecting exploitable front-running vulnerabilities in smart contracts. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 2198–2216 (2024). https://doi.org/10.1109/SP54263.2024.00146

Yang, S., et al.: Hyperion: Unveiling DApp Inconsistencies Using LLM and Dataflow-Guided Symbolic Execution. In: Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 2125–2137 (2025). https://doi.org/10.1109/ICSE55347.2025.00015

Zhang, Z., et al.: Demystifying exploitable bugs in smart contracts. In: Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 615–627 (2023). https://doi.org/10.1109/ICSE48619.2023.00061

Zheng, Z., et al.: Dappscan: building large-scale datasets for smart contract weaknesses in dApp projects. IEEE Transactions on Software Engineering 50(6), 1360–1373 (2024). https://doi.org/10.1109/TSE.2024.3383422

Zhou, L., et al.: Sok: Decentralized finance (defi) attacks. In: Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP), pp. 2444–2461 (2023). https://doi.org/10.1109/SP46215.2023.10179435